

RUTGERS

THE STATE UNIVERSITY
OF NEW JERSEY

CS520 Project 3

Tu hai kahan(Where are you)?

Sridevi M R
sm3442
Zeba Afreen
za311

May 9, 2025

1 Introduction

Imagine a robot on a spaceship that suddenly forgets where it is. It knows the layout of the ship, but a neutrino storm has knocked out its sensors—so it can't see, feel walls, or know its position. All it has is a rough idea of where it might be, represented by a set of possible locations on the map. The challenge is to figure out exactly where it is, just by trying to move around and narrowing down the possibilities. In this project, we explore strategies to help the robot localize itself as efficiently as possible. Starting with a basic rule-based method, we then use Machine learning to predict how long localization will take from different situations. These predictions guide smarter decision-making, and over time, we refine the robot's behavior through a loop of training, testing, and improving. What begins as a simple search problem gradually evolves into a reinforcement learning-inspired process aimed at smarter, faster localization.

2 Representing the Problem

a) How many possible L sets are there?
There are $2^n - 1$ possible sets of L, where n is the number of open cells and we are only considering non-empty sets.

b) For a set of locations L where the bot knows it might be, after attempting to move left, what does the bot know about where it might be? Right? Up or Down? Show that in each of these cases, the set of 'next possible locations' is well defined.
When the bot attempts to move in a particular direction (left, right, up, or down), for each location in L, if the move is allowed, then the bot moves to a new location. If there is an obstacle, then the bot remains in the same location. So, the next set of possible locations (L_{next}) is very well defined. Below is the pseudocode demonstrating the same.

Algorithm 1 Bot Move Function

```
1:  procedure BOT_MOVE(maze, locations, direction)
2:      (dx, dy) ← DIRECTIONS(direction)
3:      locations_next ← ∅
4:      for all (x, y) ∈ locations do
5:          next_x ← x + dx
6:          next_y ← y + dy
7:          if 0 ≤ next_x < DIMENSION and 0 ≤ next_y < DIMENSION
    and maze[next_x][next_y] ≠ CELL_OBSTACLE then
8:              locations_next ← locations_next ∪ {(next_x, next_y)}
9:          else
10:             locations_next ← locations_next ∪ {(x, y)}
11:     return locations_next
```

c) Show that for a map Map, there are some sets L for which C(Map,L) is easy/immediate/obvious to calculate. What are these L?

- The most immediate case that can be considered is when $|L| = 1$, where there is only one location and the bot knows exactly where it is.
- If L contains just 2 or 3 possible locations, it becomes much easier to figure out where the bot is. Since there are only a few possibilities to consider, you don't need many moves to narrow it down. With just a small number of actions, the bot can quickly eliminate the wrong options and localize itself.

d) For any general L, however, give a recursive formulation for C, in the spirit of the Bellman Equations.

In the spirit of Bellman equations, we can give the recursive formulation for C^* as follows:

- If $|L| = 1$, then $C^* = 0$, which implies that when there is only one possible location, the minimum number of actions or steps required to localize the bot is zero.
- If $|L| \neq 1$, which implies L has multiple locations, then $C^*(\text{Map}, L)$ can be defined as:

$$C^*(\text{Map}, L) = 1 + \min_{\text{direction} \in \{\text{up, down, left, right}\}} C^*(\text{Map}, L_{\text{next}})$$

e) This is not a finite-horizon problem, but it is also not an infinite-horizon problem necessarily (hence it needs no discounting). Why?

- The problem defined does not have a prior fixed number of steps required for the Bot to localize.Hence, it is not a finite horizon problem.
- This not a infinite horizon because the Bot will definitely localize in finite time as the grid is n x n, which is finite.

Discounting is not required here because every step the bot takes is equally important. Each move brings the bot closer to figuring out where it is, and all moves cost the same. Since the process naturally ends once the bot is fully localized, there's no reason to treat early steps as more valuable than later ones.

f) Give a rough argument why, for maps of our type, it is extremely unlikely, if not impossible, to have a situation where C (Map, L) is infinite.

Our map ,which is a finite dimension, which consists of open and blocked cells. The bot is considered to be in open cells. Every time the bot moves some of the locations in the belief set will either change(move to another location) or stay in the same location, Overtime, the belief set will shrink and eventually the bot will localize.The process does complete in a finite amount of time. So we can conclude that C(Map,L) is not infinite.

g) Argue that if the function C were easy to calculate, then for any given set of locations L, it would be easy to determine what the next action you should take should be.

If $C^*(\text{Map}, L)$ were easy to compute, choosing the bot's next move would be simple.

For example, let's say the bot could be in one of three locations: $L = \{(2,4),(5,4),(6,8)\}$.

By simulating each possible move (up, down, left, right), we can compute $C^*(\text{Map}, L_{\text{next}})$ for each resulting belief set and choose the move that leads to the lowest cost.

This will make the decision process easy and particularly effective.

h) How could you apply our graph search techniques to this problem?
Each belief set L can be represented as a node and every time the bot makes a move(up,down,left,right), The belief set gets updated to a new belief set forming edges between these nodes. The goal is to make the belief set $|L| = 1$ by taking minimum number of steps which implies that the bot is localized. From the representation mentioned above we can clearly draw a conclusion that a graph search technique will hold good to this problem.

i) What might a good heuristic for A be here? Note: does not have to be consistent or admissible, it just needs to meaningfully estimate work remaining.

A good heuristic here can be the belief set size minus 1, which can help give the bot an estimate of how many steps remain for it to localize ($|L| = 1$).

This heuristic is not particularly admissible, but it serves its purpose by reflecting the amount of work remaining for localization.

The belief set essentially indicates that the bot might be in different positions at a given time, and when the belief set is large, the bot's uncertainty about its true position increases.

This heuristic does help narrow down the positions, even though it's not quite accurate.

j) How should we understand something like A, relative to the problem of finding C above? Are they completely separate approaches to the problem, or is there a connection between them? Be clear and explicit.

C^* is the minimum number of steps required for the bot to localize. A* helps find this by exploring different belief sets and using a heuristic to estimate how many steps are remaining for the bot to localize. It guides the bot toward actions that reduce the number of possible locations and eventually reaching $|L| = 1$. This, in turn, helps compute $C^*(L)$ efficiently. Hence, we can say there is a connection between them.

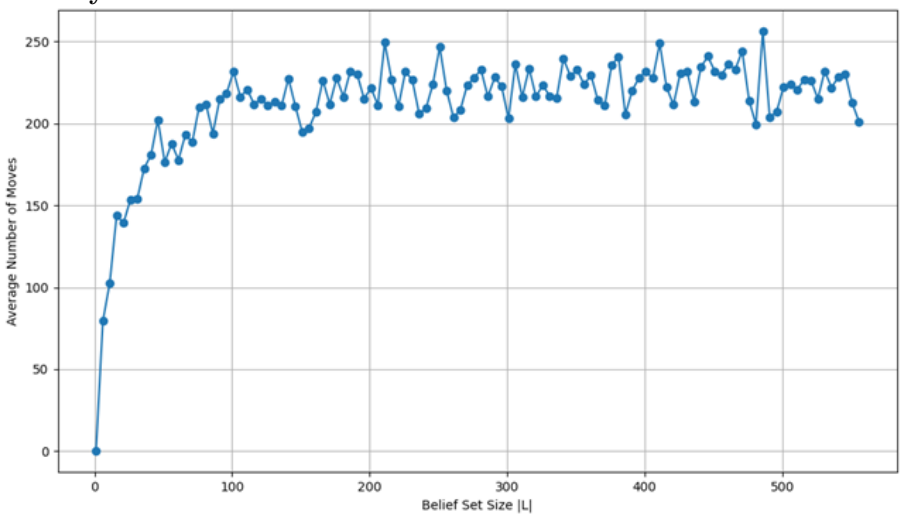
3 A Solution Emerges.

To implement the π_0 strategy, an algorithm was designed to simulate the bot's movement toward either a designated target or a dead-end. The algorithm tracks the evolving belief state at each step and terminates once the bot is fully localized.

Algorithm 2 RunLocalizationExperiment

```
procedure RunLocalizationExperiment(maze, initial_locations)
    L ← set of initial_locations
    total_moves ← empty list
    targets ← FindCornersOrDeadEnds(maze)
    if targets is empty then
        raise Exception
    target ← random choice from targets
    while |L| > 1 do
        start ← random choice from L
        path ← AStar(maze, start, target)
        if path is empty then
            break
        for all direction_index in path do
            direction ← direction corresponding to direction_index
            L ← bot_move(maze, L, direction)
            Append direction to total_moves
            if |L| = 1 then
                break
    return total_moves
```

Question: Create a plot with $|L|$ on the x-axis (running from 1 to the total number of open cells), and *AverageNumberOfMovesNeeded* on the y-axis.



This graph shows the average number of moves needed to localize the bot using π_0, based on the size of the belief set $|L|$. For each value of $|L|$, 50 random trials were run, and the results were averaged. The plot includes every 5th point to reduce clutter. As $|L|$ increases, the number of moves rises quickly at first, then gradually levels off. This indicates that while larger belief sets make localization harder, the increase in difficulty slows down beyond a certain point. This trend highlights the limitations of the random-move strategy used in π_0.

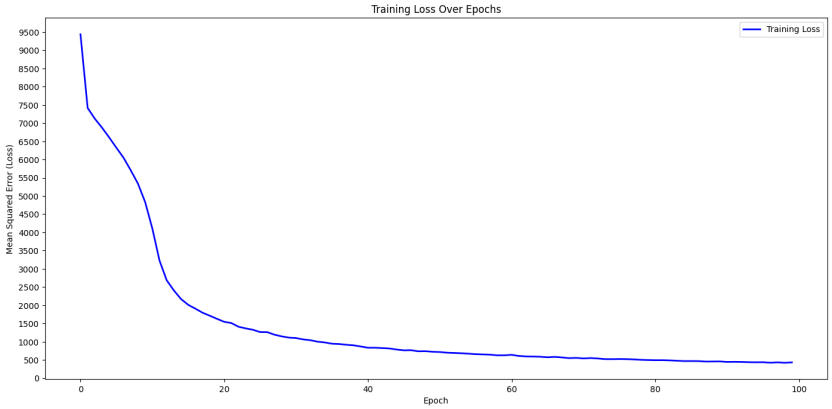
4 What Does This Have To Do With Machine Learning, Dr. Cowan?

- **Input Space:** The input space L in our model is represented has a grid with obstacles marked as 1, open cells marked as 0 and the possible bot positions marked as 0.5.
- **Output Space:** The output space is predicted is a real value which represents the predicted number of steps the bot would take to localize.
- **Model Space:** The model space is designed as follows
  - Two convolutional layers:
    - The first layer has 16 filters with a 3×3 kernel and ReLU activation.
    - The second layer has 32 filters, also with a 3×3 kernel and ReLU activation.
  - Max pooling is applied after each convolution to reduce spatial dimensions.
  - The output is flattened and passed through two fully connected layers:
    - First dense layer: 64 neurons with ReLU activation.
    - Output layer: 1 neuron to predict a single real-valued number (the number of steps remaining to localize the bot).
    - A final ReLU activation is applied to ensure the prediction is non-negative.
- **Loss Function:** The loss function is Mean Squared Error loss (MSE) implemented using `nn.MSELoss()`. This function measures the average squared difference between the predicted number of steps and the actual number of steps needed for the bot to localize.
- While, MSE does'nt particularly prevent negative output. In order to include this constraint our model has a ReLU activation in the final layer to make sure there are no non-negative outputs.
- **Training Algorithm:** The training algorithm uses batch-based gradient descent with the Adam optimizer to minimize Mean Squared Error (MSE) between the predicted and actual number of steps to localization. The model is trained for 100 epochs with a learning rate of 0.0005 and a batch size of 64. In each batch, the model makes predictions, calculates how far off they are, and adjusts itself to do better next time. The dataset is split into 80% for training and 20% for testing, and the loss is recorded after each epoch to track how the model is learning.

Question: Be clear and specific as to how you generated your training (and testing) data, and how much training (and testing) data you used.
To create the training and testing data, we ran a bot localization strategy on 200 randomly generated mazes. For each maze, the bot was given a set of possible starting positions, sometimes using all open cells (full belief) and sometimes using 75% of them (partial belief). In total, we generated 47,570 samples. We used 80% of them to train the model and 20% to test.

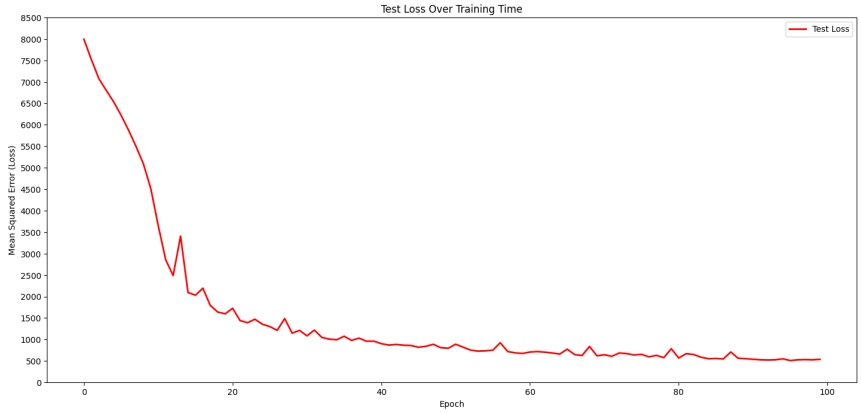
Question:Be clear about any steps you took to reduce overfitting.
To reduce overfitting, we started by splitting the data into training and testing sets (80/20), so we could check how well the model was doing on unseen examples. We also shuffled the data during training and used in batches of 64 to help the model learn general patterns.

Question:Generate a plot of training loss (per data point) over training time (to verify that the model is learning, and that loss is decreasing).



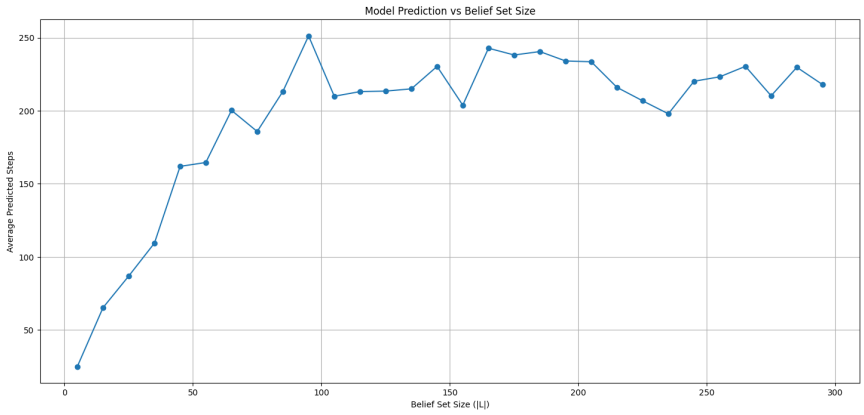
The above graph shows Training loss over 100 Epochs. As seen in the graph the loss is decreasing steadily as our model is learning from the training data. Which implies that our model is improving its predictions overtime.

Question:Generate a plot (on the same axes) of testing loss (per data point) over training time (to verify the model is generalizing well and not overfitting).



The above graph shows Testing loss over 100 Epochs. As our model learned, the loss on the unseen 20% data decreased steadily, showing it has learned pretty well from training data.

Question:Once your model is trained, create a plot with $|L|$ on the x-axis, average predicted number of moves on the y-axis, and for each $|L|$, generate a number of sets L, evaluate your model on each of them, and average the results for that $|L|$ together. How does this graph compare to the one from the previous section?.

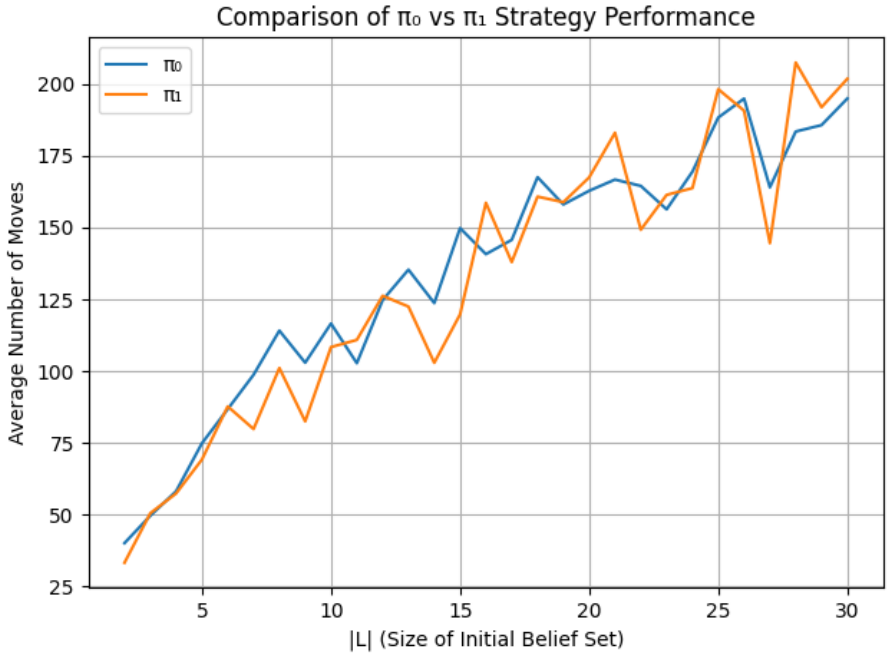


The plot above shows the average number of moves predicted by the CNN model for various belief set sizes . For each size, 20 different belief sets were sampled and evaluated using the model. From the graph we can see that the model pre-

dicts more steps when the belief set size is higher. When we compare it to our section 2 algorithm the CNN model does give a lower prediction count. Which means that our model is just not copying the steps like our algorithm but its actually learning to predict more efficient paths.

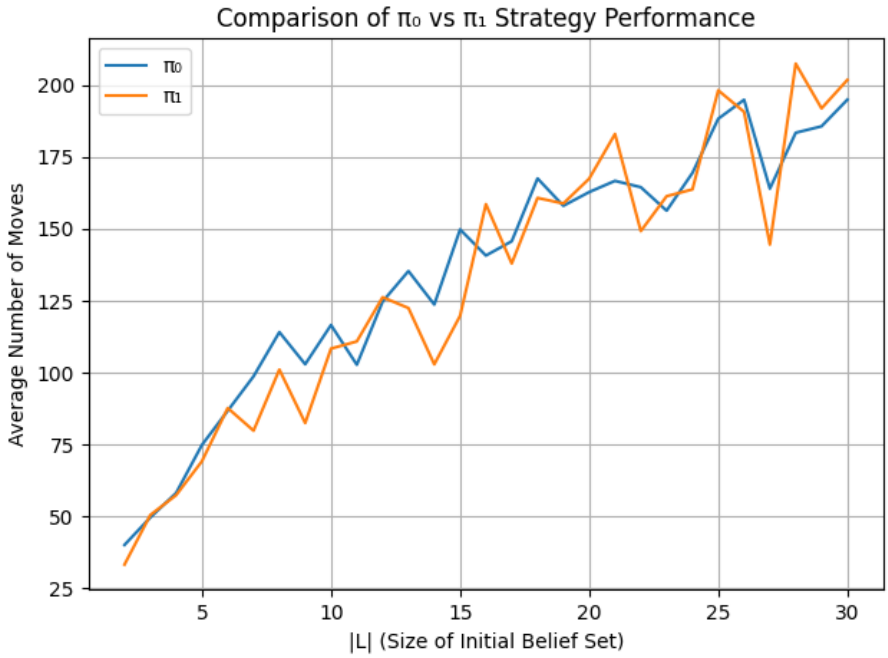
5 Improving Your Strategy?

Using the trained estimator $\hat{C}_0$, which was used to estimate the number of steps required to localize the bot under policy π_0, we created a new policy called π_1 that improves on π_0 by making a smarter first move. From the initial belief set L, the bot tries out all four possible directions in simulation and uses $\hat{C}_0$ to predict which one would lead to the fastest localization. It then picks the direction with the lowest predicted cost, makes that move, and continues the rest of the process using the original π_0 strategy. The below graph shows the comparison of performance between π_0 and π_1.



This plot compares how the original strategy (π_0) stacks up against the improved version (π_1), which uses a one-step lookahead based on predicted localization costs. On the x-axis, we have the size of the belief set and, on the y-axis, the average number of moves it takes to localize. You can see that π_1 usually does better, especially when the bot starts off with a lot of uncertainty. By making a smarter first move, π_1 helps the bot localize faster in many cases. This shows that even a small dose of predictive guidance can make a noticeable difference.

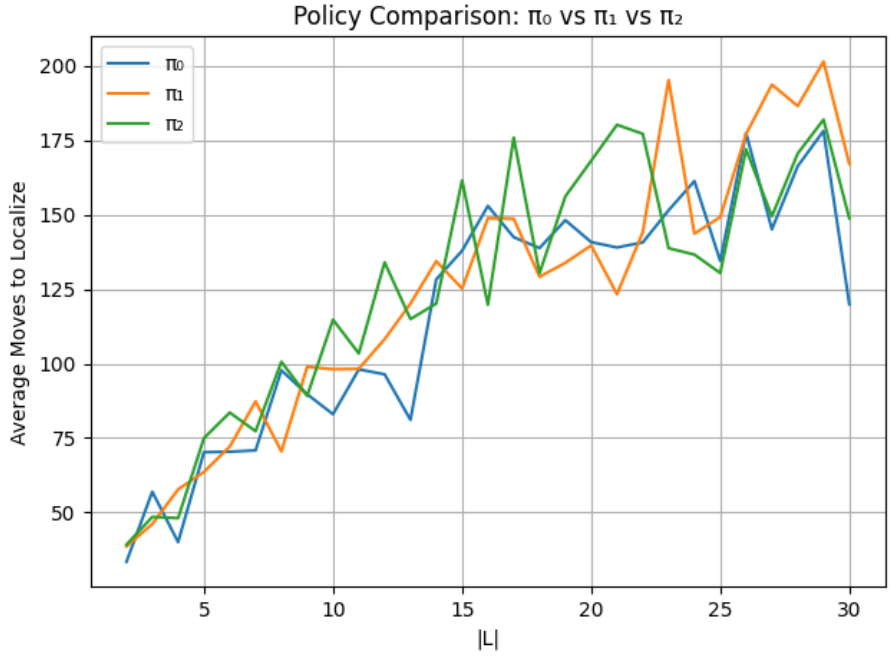
To improve the strategy further, a new dataset was collected using policy π_1, which chooses the best first move and then follows π_0. The new model trained is $\hat{C}_1$. The below graph depicts the Training loss vs the Test loss.



This graph shows the training and test loss curves for the model, which was trained to predict the localization cost under policy π_1. The x-axis represents the training epoch, and the y-axis shows the mean squared error (MSE) loss. As seen in the plot, both the training and test losses drop significantly during the first few epochs, then gradually level off. The close alignment between the two curves suggests that the model is learning effectively without overfitting. Although the test loss fluctuates slightly, it remains well-aligned with the training loss, indicating that the model generalizes well to unseen belief sets.

To make the strategy even better, policy π_2 was built. The policy π_2 builds on π_1 by making an even smarter first move. Starting from the current belief set L, the bot tries out all four directions and uses the improved model $\hat{C}_1$ to predict the cost for each. It then picks the move with the lowest predicted cost and makes that move. After that, it follows the π_1 strategy, which already uses $\hat{C}_0$ for its first step and then continues with π_0.

The below graph depicts the performances of all the three policies.



The graph shows how π_0, π_1, and π_2 perform as the size of the belief set increases. For smaller belief sets (uncertainty is less), all three policies perform similarly. As $|L|$ increases, π_1 and π_2 perform a little better than π_0, as they make more informed and smart first moves. Among them, π_2 performs slightly better.

Question : Why is generating novel data each time you update the policy necessary?

It is absolutely important to generate new data each time a policy is updated because, different policies will lead the bot through different situations and belief sets. If a model is trained on data from one policy , it might not work well for another, since it has not seen the types of states that new policy goes to. By creating new data time each time the model is made to learn from new fresh samples, which leads to variety and avoid redundancy.